

[← Back to CCDV-F Sample Tests](#)

Great Job!

Applications and Integration Practice Test Complete

10/10

Correct

100%

Score

1:48

Time



This was a sample test from a limited pool of questions.

Upgrade to Premium for the complete CCDV-F bank with hundreds more questions!

Upgrade to Premium

Get unlimited access to:

- ✓ Unlimited practice tests across all domains
- ✓ Full-length 53-question mock exams with realistic timing
- ✓ Detailed explanations and study guides
- ✓ Progress tracking and performance analytics
- ✓ Smart practice focusing on weak areas

Review Your Answers

1

[→ Login to Clarify](#)

A team needs to summarize 120,000 archived support tickets each night. No user is waiting for an

immediate response, and the results only need to be available in the data warehouse by 8 a.m. Which integration approach is the best fit?

A.
Use synchronous Messages API calls from the web tier for each ticket.

B.
Use the Message Batches API for the overnight processing job.



C.
Use streaming responses so every summary starts displaying immediately.

D.
Keep one long conversation session open for all ticket summaries.

Explanation:

Correct answer (B): The Message Batches API is best for large numbers of requests that do not require immediate, user-facing responses. This workload is high-volume, latency-tolerant, and scheduled as an offline job, so batch processing matches the operational requirement better than synchronous realtime calls.

Why the other options are wrong:

- Option A: Synchronous Messages API calls are appropriate for interactive or latency-sensitive flows, but this workload is offline and high volume.
- Option C: Streaming improves perceived responsiveness when a user is waiting, but it does not make an overnight batch job a better fit.
- Option D: A single long session would mix unrelated tickets and create poor session boundaries rather than matching the batch workload.

2

[→ Login to Clarify](#)

A customer-facing writing assistant often produces responses that take 8 to 12 seconds. Product wants users to see useful text as soon as it is generated, while still validating the final response before saving it. What is the most appropriate integration change?

A.

Enable streaming and render partial output while retaining final response handling.



B.

Switch the feature to Message Batches API for all user requests.

C.

Remove response validation because streaming already confirms correctness.

D.

Concatenate all prior user sessions to make output start faster.

Explanation:

Correct answer (A): Streaming is an integration pattern for receiving partial output progressively, which improves responsiveness in user-facing interactions. The application should still handle and validate the final response because streaming changes delivery behavior; it does not guarantee correctness or remove downstream integration responsibilities.

Why the other options are wrong:

- Option B: Batch processing is for requests that do not need immediate user-facing responses, so it conflicts with the interactive product goal.
- Option C: Streaming changes delivery timing, not correctness guarantees; final response handling and validation are still needed.

- Option D: Adding unrelated session history creates stale context and is not a valid way to improve progressive rendering.

3

[→ Login to Clarify](#)

A production incident review finds that two app instances produced different support-case classifications after a rolling deploy. The configuration differs as follows:

Instance A: model = "latest", prompt loaded from admin UI draft Instance B: model = "2024-locked", prompt loaded from Git tag v18

The team's priority is reproducible behavior and rollback control. What is the best next change?

A.

Move model identifiers and prompt versions into reviewed, version-controlled configuration.



B.

Ask operators to paste the current prompt into each instance during deployment.

C.

Use only the SDK default model so all instances share provider defaults.

D.

Store production prompts in chat history so recent edits are always available.

Explanation:

Correct answer (A): Production Claude applications should explicitly manage model versions, prompt versions, settings, and dependencies so behavior can be reviewed, reproduced, rolled back, and maintained across environments. Version-controlled configuration is the best fit for the

stated goal of reproducibility and rollback control.

Why the other options are wrong:

- Option B: Manual prompt pasting is error-prone and makes review, rollback, and reproducibility worse.
- Option C: Provider or SDK defaults can change or differ by environment, which undermines the stated reproducibility goal.
- Option D: Chat history is not durable application configuration and does not provide controlled deployment or rollback.

4 

[→\] Login to Clarify](#)

A team migrates from direct REST calls to a cloud provider's Claude SDK. Their downstream parser starts failing because it assumed the response body was a plain string. The new wrapper returns:

```
{ "id": "msg_123", "content": [{"type": "text", "text": "Approved"}], "stop_reason": "end_turn" }
```

What should the developer do first?

A.

Update the integration to handle the structured message response contract.



B.

Remove JSON parsing and treat the entire response object as final text.

C.

Switch back to REST because SDKs cannot preserve Claude response semantics.

D.

Send a longer prompt asking Claude to return a simpler SDK object.

Explanation:

Correct answer (A): SDKs and third-party provider integrations can abstract transport details, but developers still need to understand and handle the underlying Claude request and response structure. The parser should be updated to consume the structured response contract rather than assuming a plain string.

Why the other options are wrong:

- Option B: Treating the whole object as text ignores the response structure and would likely pass metadata to downstream code incorrectly.
- Option C: The problem is not that SDKs cannot work; it is that the integration must handle the SDK's structured response correctly.
- Option D: Prompting cannot change the wrapper's object schema; the application must parse the response contract it receives.

5 [→ Login to Clarify](#)

A legal research app keeps appending every prior matter a lawyer worked on into each new Claude request. Users report that summaries sometimes mention unrelated clients from earlier sessions. The product requirement is to summarize only the currently opened matter. What is the best application-level fix?

A.

Create each request from the current matter's relevant data and clear unrelated session context.



B.

Increase the maximum context sent so Claude can infer which matter is current.

C.

Move all prior matters into the system prompt so they are treated as durable context.

D.

Enable streaming so unrelated references can be noticed earlier by the user.

Explanation:

Correct answer (A): Session hygiene is an application responsibility. The app should construct requests with only the relevant context for the current task and avoid carrying stale or unrelated content across sessions. This preserves content boundaries and reduces cross-session contamination.

Why the other options are wrong:

- Option B: Sending more stale context makes the boundary problem worse and relies on the model to infer application state.
- Option C: Putting unrelated matters in durable instructions would further mix data that should be kept out of the current request.
- Option D: Streaming affects response delivery timing, not whether the request contains irrelevant prior-session context.

6

[→ Login to Clarify](#)

A claims-processing service sends Claude-generated results directly to a database writer. The writer expects this contract:

```
{ "claim_id": "string", "decision": "approve | deny | review", "rationale": "string" }
```

Recent failures occurred when the response included prose before the object. What is the best integration change?

- A.
Define and validate a structured output contract before passing data to the writer.



- B.
Ask the database writer to accept any natural-language explanation as valid input.

- C.
Store the raw response first and let analysts manually extract fields later.

- D.
Append more prior examples until Claude always emits the exact object.

Explanation:

Correct answer (A): Downstream code needs predictable contracts, validation expectations, and clear separation between generated content and application data. The integration should define the expected structure and validate it before sending data to the database writer, rather than relying on informal prose formatting.

Why the other options are wrong:

- Option B: A database writer that accepts arbitrary prose would remove the predictable contract needed by the application.
- Option C: Manual extraction does not solve the integration reliability problem for an automated service.
- Option D: Examples may help, but relying on examples alone is not as robust as enforcing and validating the application contract.

7

[→ Login to Clarify](#)

An insurance app must answer questions about damage visible in uploaded vehicle photos and then populate a repair-intake form. Which implementation choice best fits the application requirement?

A.

Send the relevant photos with the user task and map the response to the form contract.



B.

Send only the photo filenames and ask Claude to infer the visual details.

C.

Upload every photo in the customer account to maximize available context.

D.

Convert the request to batch processing because images are present.

Explanation:

Correct answer (A): A vision-capable application flow should include the visual content needed for the task, keep boundaries clear, and connect the result to the downstream application contract. Sending the relevant images with the task supports the requirement while avoiding unrelated content.

Why the other options are wrong:

- Option B: Filenames do not provide the visual content needed to answer questions about visible damage.
- Option C: Sending every account photo adds unrelated context and weakens application boundaries.
- Option D: The presence of images does not by itself determine realtime versus batch; the

user task and latency requirements do.

8



→ Login to Clarify

A compliance team needs to summarize 80,000 archived support tickets every night. No user waits for an individual response, and results are loaded into a reporting database by 6 a.m.

Artifact:

```
{  
  "job": "nightly_ticket_summaries",  
  "items": 80000,  
  "deadline_hours": 8,  
  "interactive_user_waiting": false,  
  "current_plan": "send one synchronous Messages API request per ticket"  
}
```

What is the best integration pattern for this workload?

A.

Submit the tickets through the Message Batches API and process results asynchronously.



B.

Keep synchronous Messages API calls and raise the HTTP client timeout.

C.

Enable streaming on every request and concatenate streamed tokens in the database.

D.

Combine all tickets into one large Messages API request to reduce call count.

Explanation:

Correct answer (A): The workload is high-volume and latency-tolerant, with no user waiting for an immediate answer. In the Applications and Integration domain, this is the classic fit for Message Batches API rather than many realtime calls. Batch processing also supports an operational workflow where results can be collected later and loaded into downstream systems.

Why the other options are wrong:

- Option B: Longer timeouts do not change the workload characteristics and can make reliability, retry handling, and operations worse.
- Option C: Streaming helps user-facing perceived responsiveness, but this job has no interactive user and would still require managing many realtime calls.
- Option D: Putting all tickets into one request risks context and data-boundary problems and does not create a manageable per-ticket processing workflow.

9



→ Login to Clarify

A developer is building a customer chat UI. Users abandon the page if nothing appears for several seconds, but the team does not need streaming to change the final answer quality.

Artifact:

```
{
  "messages": [
    {"role": "user", "content": "Explain my invoice charges."}
  ],
  "stream": false
}
```

Which change best addresses the product requirement?

A.

Enable streaming and update the UI as partial output arrives.



B.

Use the Message Batches API and poll until the answer is complete.

C.

Store the full conversation in local browser storage before each request.

D.

Ask Claude to answer faster by adding a short instruction to the user message.

Explanation:

Correct answer (A): Streaming is an integration behavior that lets the application receive partial output as it is generated. For an interactive chat UI, streaming improves perceived responsiveness and lets the UI show progress before the final response is complete. It should be implemented with asynchronous handling of partial events; it does not by itself make the answer more accurate.

Why the other options are wrong:

- Option B: Batch processing is intended for high-volume latency-tolerant work, not a live chat where a user is waiting.
- Option C: Browser storage does not solve delayed first output and can introduce session-management problems if used carelessly.
- Option D: A prompt instruction may influence verbosity or style, but it is not the reliable integration mechanism for incremental delivery.

10 [→ Login to Clarify](#)

A SaaS support assistant sometimes references another customer's ticket. A code review finds this request builder:

Artifact:

```
const sharedHistory = [];  
  
function buildRequest(userId, newText) {  
  sharedHistory.push({ role: "user", content: newText });  
  return { messages: sharedHistory };  
}
```

What is the best fix?

- A.
Keep a separate application-side message history per user or session.



- B.
Add a system instruction saying never mention other customers' tickets.

- C.
Clear the shared history only when the process restarts each night.

- D.
Ask Claude to summarize the shared history before every new request.

Explanation:

Correct answer (A): Application code must maintain durable state and session records outside the

model context, and unrelated users or tenants must be separated. The bug is caused by shared conversation state being reused across users. A per-user or per-session history store fixes the integration boundary instead of relying on the model to infer isolation.

Why the other options are wrong:

- Option B: An instruction may reduce some bad behavior, but it cannot correct an application bug that includes another user's context in the request.
- Option C: Nightly clearing still mixes users during the day and does not provide reliable session isolation.
- Option D: Summarizing shared history preserves the core contamination problem because unrelated user data is still in the request path.